

Technical Report — PM10 Air Quality Forecasting for Lombardy

Course: Data Science Exam Project

Dataset period: 2024-01-01 → 2026-05-07

Last model run: 2026-06-02

Language: Python 3.11

Repository root: `Data-Science-Exam-Project/`

Table of Contents

1. Project Overview
2. Domain Context and Motivation
3. System Architecture
4. Step 1 — Data Collection
5. Step 2 — Data Ingestion and Storage
6. Step 3 — Exploratory Data Analysis and Feature Engineering
7. Step 3.5 — Random Matrix Theory Diagnostic
8. Step 4 — Regression
9. Step 5 — Classification
10. Step 6 — Station Clustering
11. REST API and Web Interface
12. Cloud Deployment
13. Data Quality and Freshness Management
14. Results Summary
15. Design Decisions Log

1. Project Overview

This project builds an end-to-end data science pipeline for **daily PM10 air quality forecasting** across **67 monitoring stations in Lombardy, Italy**. Given the strong public health and regulatory relevance of particulate matter pollution in the Po Valley — one of Europe's most polluted regions — the system addresses three distinct machine learning tasks:

1. **Regression:** predict next-day PM10 concentration in $\mu\text{g}/\text{m}^3$ (continuous target).
2. **Classification:** predict next-day alert class (4-class ordinal: `verde`, `giallo`, `arancio`, `rosso`) derived from WHO/EU thresholds applied to PM10.
3. **Clustering:** group the 67 stations into typologies based on their long-run pollution profiles, independently of the supervised tasks.

Beyond the modelling core, the project delivers a **production-ready serving layer**: a FastAPI web application (with a Jinja2 HTML frontend) containerised with Docker and deployed on **Google Cloud Run**. The serving path reads recent weather and pollutant data on demand from **Google Cloud Storage**, bypassing any local database dependency.

The pipeline is designed to be fully reproducible (fixed seeds, deterministic temporal splits) and to avoid all forms of data leakage — a critical requirement when working with time-series data that spans multiple stations.

2. Domain Context and Motivation

2.1 Why PM10?

PM10 (particulate matter with aerodynamic diameter $\leq 10\text{ }\mu\text{m}$) is the principal regulated pollutant in the Po Valley. The region is geographically enclosed by the Alps to the north and west and the Apennines to the south, creating a natural basin that traps pollutants under stable atmospheric conditions. Lombardy exceeds the EU daily limit ($50\text{ }\mu\text{g}/\text{m}^3$, not to be exceeded more than 35 days/year) at many stations, especially in autumn and winter when thermal inversions suppress vertical mixing.

Early-warning forecasts allow both public authorities and citizens to anticipate high-pollution days, enabling timely interventions (e.g., traffic restrictions, heating bans) and protective behaviour.

2.2 Alert Class Definition

The four-class alert system used throughout this project is defined as instantaneous daily thresholds applied to measured PM10:

Class	PM10 range ($\mu\text{g}/\text{m}^3$)	Regulatory reference
verde (green)	[0, 20)	WHO AQG 2021 annual guideline
giallo (yellow)	[20, 35)	Intermediate target level
arancio (orange)	[35, 50)	Buffer zone below EU daily limit
rosso (red)	[50, ∞)	Exceeds EU 2008/50/CE daily limit

Important design note: these thresholds define an *instantaneous* daily classification for modelling purposes. They are distinct from the *operational anti-smog alert* system of the Lombardy Regional Government (DGR 449/2018), which activates after N consecutive days above $50\text{ }\mu\text{g}/\text{m}^3$. The instantaneous approach is deliberately chosen to produce a target that is balanced across the dataset and suitable for supervised multi-class classification, whereas the consecutive-day rule would produce an extremely rare positive class.

2.3 Data Sources

Source	Type	Coverage	API
ARPA Lombardia (Socrata)	Air quality measurements	2024-01-01 → 2026-05-07	dati.lombardia.it/resource/nicp-bhqi.json

Source	Type	Coverage	API
ARPA Sensor Registry	Station/sensor metadata	Static	<code>dati.lombardia.it/resource/ib47-atvt.json</code>
Open-Meteo (historical forecast)	Hourly meteorology	2024-01-01 → 2026-05-07	<code>historical-forecast-api.open-meteo.com/v1/forecast</code>
OpenStreetMap (Overpass API)	Industrial zone polygons	Static	Fetches via <code>fetch_industrial_zones.py</code>

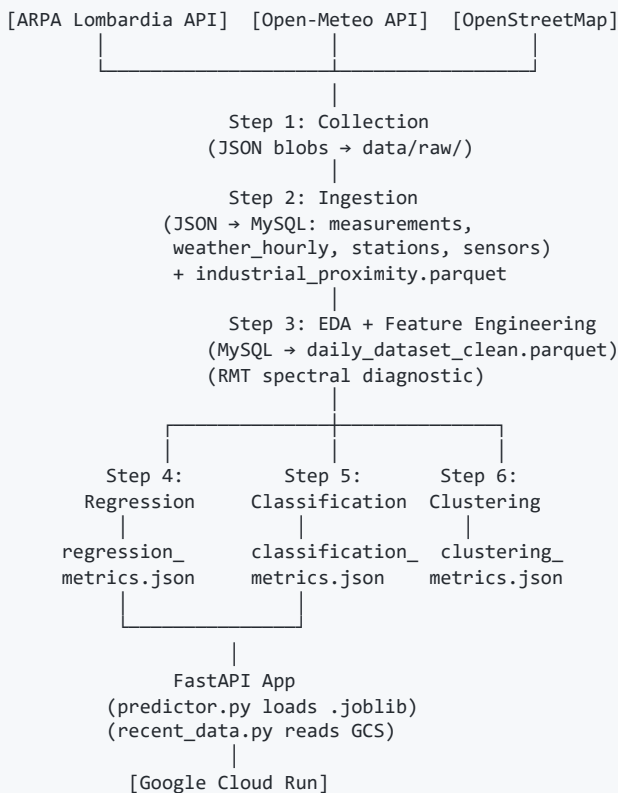
3. System Architecture

The project is organised as a set of sequentially dependent Python packages, each corresponding to one step of the pipeline. All steps share a common `shared/` utility module.

```
Data-Science-Exam-Project/
├── step_1_collection/           # Data collection from ARPA + Open-Meteo → JSON blobs
│   ├── collector.py           # Core fetch functions (measurements, weather)
│   ├── backfill.py           # Historical backfill CLI
│   ├── fetch_industrial_zones.py
│   ├── gcloud_backfill/      # Cloud Run Job: historical backfill
│   └── gcloud_daily/         # Cloud Run Job: daily rolling refresh
├── step_2_ingestion/          # ETL: JSON blobs → MySQL (training store)
│   ├── ingest.py             # Orchestrator (stations, sensors, measurements, weather)
│   ├── schema.sql            # MySQL schema definition
│   ├── spatial.py            # Industrial proximity computation (GeoPandas)
│   └── compose.yaml          # Docker Compose for local MySQL stack
├── step_3_eda/                # Feature engineering, EDA, dataset construction
│   ├── build_dataset.py      # Joins PM10 + weather + pollutants, adds features
│   ├── config.py             # Thresholds, constants, plot settings
│   ├── db.py                 # MySQL query helpers
│   ├── eda.py                # Main EDA orchestrator (plots, analysis, RMT)
│   ├── rmt.py                # Random Matrix Theory spectral diagnostics
│   ├── plots.py              # All 16 EDA visualisations
│   ├── analysis.py           # Statistical summaries
│   ├── daily_dataset_clean.parquet # Intermediate artifact (ML input)
│   └── plots/                # 16 PNG plots
├── step_4_regression/          # Regression: ElasticNet, XGBoost, RandomForest
│   ├── config.py             # Hyperparameter grids, CV settings
│   ├── train.py              # Pipeline factories + GridSearch/RandomizedSearch
│   ├── evaluate.py           # Metrics, residual plots, feature importance
│   └── artifacts/            # Saved models (.joblib) + metrics JSON
├── step_5_classification/      # 4-class alert classification
│   ├── config.py             # Label encoding, hybrid strategy parameters
│   ├── train.py              # Train 3 classifiers + ordinal
│   ├── evaluate.py           # Classification report, confusion matrix
│   ├── calibrate.py          # Probability calibration (isotonic)
│   ├── ordinal_classifier.py  # Frank & Hall (2001) ordinal decomposition
│   ├── train_ordinal.py      # Train OrdinalClassifier wrapper
│   ├── evaluate_hybrid.py    # Hybrid strategy evaluation
│   ├── tune_hybrid.py        # Hybrid parameter search (OOF)
│   ├── guardrails.py         # Hard constraint checking (recall_rosso, SER)
│   └── artifacts/            # Saved models + metrics JSON
├── step_6_clustering/          # Unsupervised station clustering
│   ├── config.py             # Feature set, algorithm parameters
│   ├── profiles.py           # Build per-station aggregate profiles
│   ├── cluster.py            # KMeans, Agglomerative, DBSCAN grid search
│   ├── evaluate.py           # Internal + external validation, plots
│   ├── map_view.py           # Folium cluster map
│   └── artifacts/            # Cluster labels, metrics, cluster_map.html
└── api/                       # FastAPI application (backend + frontend)
```

└─ main.py	# App entry point, routing, lifespan hooks
└─ schemas.py	# Pydantic response models
└─ routers/	# /stations, /forecast, /history, /clusters
└─ services/	# predictor.py, history.py, recent_data.py, ...
└─ templates/	# Jinja2 HTML (map, forecast, history)
└─ static/	# CSS, static assets
└─ shared/	# Cross-step utilities
└─ utils.py	# temporal_train_test_split, build_preprocessor, ...
└─ data/raw/	# Raw JSON blobs (training archive, immutable)
└─ Dockerfile	# Container for Cloud Run deployment
└─ requirements.txt	# Python dependencies
└─ technical_report.md	# This document

3.1 Data Flow Diagram



4. Step 1 — Data Collection

Directory: `step_1_collection/`

Key files: `collector.py`, `backfill.py`, `gcloud_backfill/`, `gcloud_daily/`

4.1 Objectives

The collection layer is responsible for downloading raw air quality measurements and meteorological data for every day in the study period, and persisting them as JSON files. The design deliberately separates collection from ingestion: raw blobs are immutable once written (with the exception of the weekly heal job, described in §13) and serve as a reproducible archive.

4.2 ARPA Lombardia Measurements

The ARPA Lombardia open data portal exposes air quality measurements through the Socrata API. The primary endpoint is:

```
https://www.dati.lombardia.it/resource/nicp-bhqi.json
```

A SoQL `$where` clause filters records to a single calendar day and — for the historical archive — to validated measurements only (`stato = 'VA'`):

```
where_clause = (  
  f"data >= '{date_str}T00:00:00.000' "  
  f"AND data <= '{date_str}T23:59:59.999' "  
  f"AND stato = 'VA' "  
)
```

The `stato = 'VA'` filter is critical: ARPA marks freshly uploaded measurements as `stato = 'NA'` (not yet validated), and replaces them with validated values approximately 12 working days later. Using unvalidated values for training would introduce systematic bias, because preliminary records often carry the sentinel value `-9999` for pollutants that could not be measured in time.

Pollutant resolution: PM10 and PM2.5 are reported as a single daily value at midnight (`T00:00:00`). NO₂, O₃, and CO are reported at hourly resolution. All are stored in the same `measurements` table and later aggregated during dataset construction.

Pagination: the Socrata API returns at most 50,000 records per call. The collector implements an offset-based pagination loop that exits when the returned page is smaller than the page size, ensuring complete retrieval even for high-traffic days.

Sensor registry: station and sensor metadata (name, municipality, province, coordinates, altitude, active period) is fetched from a separate endpoint (`ib47-atvt.json`) once and cached locally. This avoids redundant API calls across the hundreds of collection days.

4.3 Open-Meteo Weather Data

Meteorological data is sourced from the Open-Meteo historical forecast API, which uses the same numerical weather model as its live forecast endpoint. This is a deliberate design choice: it guarantees that the training distribution of meteorological features is consistent with what the API will use at inference time.

The following 11 hourly variables are collected per station per day:

Variable	Unit	Role in modelling
<code>temperature_2m</code>	°C	Temperature inversion proxy
<code>relative_humidity_2m</code>	%	Atmospheric moisture
<code>dew_point_2m</code>	°C	Condensation / fog proxy
<code>precipitation</code>	mm	Wet deposition (PM scavenging)
<code>surface_pressure</code>	hPa	Anticyclonic stagnation indicator
<code>cloud_cover</code>	%	Radiation/photolysis
<code>wind_speed_10m</code>	m/s	Horizontal dispersion
<code>wind_direction_10m</code>	°	Prevailing transport direction

Variable	Unit	Role in modelling
<code>visibility</code>	m	Fog/haze proxy
<code>shortwave_radiation</code>	W/m ²	Photochemical O ₃ formation proxy
<code>boundary_layer_height</code>	m	Vertical mixing depth (key for PM)

Coordinate deduplication: with ~170 sensors spread across ~67 stations, many stations share the same Open-Meteo grid cell (resolution $\approx 1^\circ$, ≈ 11 km). Rounding coordinates to one decimal place before API calls reduces the number of unique requests from ~67 to ~15–20, significantly lowering both latency and API load. The resulting hourly JSON is stored per station by interpolating the cached grid-cell response.

Retry logic: transient HTTP failures are handled with exponential backoff (initial wait 3 s, doubled on each retry, up to 3 attempts). This makes the collection robust to rate-limiting by the free-tier Open-Meteo endpoint.

4.4 Industrial Zone Proximity

A supplementary static feature captures the proximity of each monitoring station to industrial areas. OpenStreetMap industrial land-use polygons are downloaded via the Overpass API (`fetch_industrial_zones.py`) and stored as a GeoJSON file. The spatial computation (§5.4) produces one per-station feature:

- `dist_industrial_km`: Euclidean distance to the nearest industrial zone boundary.

Note on a dropped feature. An earlier `n_industrial_zones_15km` (count of distinct industrial zones within a 15 km radius) was removed after analysis. Permutation importance ranked it near the bottom in both the regressor and the classifier ($\Delta R^2 \approx 0.0005$; $\Delta f1_macro \approx 0.0004$, statistically indistinguishable from noise), and a with/without retrain confirmed every metric moved by ≤ 0.02 with mixed sign. The count merely reflected the granularity of OSM `landuse=industrial` polygon fragmentation rather than a physical quantity, so it was dropped. `dist_industrial_km` is retained — it carries a small but consistent signal and acts as a station-context proxy correlated with baseline PM10 (not a causal measure of industrial exposure).

4.5 Cloud Backfill and Daily Refresh

For production use, collection is automated via two Google Cloud Run Jobs:

- `gcloud_backfill/`: a one-time (or on-demand) job that fills any date range into a Google Cloud Storage bucket (`gs://exam-project-backfill/data/raw/`). Each day produces two blobs: `YYYY-MM-DD_measurements.json` and `YYYY-MM-DD_weather.json`. A GCS-based checkpoint prevents redundant re-downloads. A **heal mode** (`OVERWRITE_EXISTING=true`) re-downloads a rolling `[today-21, today-15]` window every Saturday at 02:00 (Europe/Rome) to replace preliminary blobs with fully validated data.
- `gcloud_daily/`: a lightweight daily job that maintains a fresh 7-day rolling window on GCS, used exclusively by the API serving path. By default it force-refreshes the last 3 days (configurable via `FORCE_REFRESH_LAST_N_DAYS`) to close gaps introduced by the ARPA preliminary-data lag.

The separation of training data (MySQL, immutable archive) from serving data (GCS rolling window) is a key architectural choice: it prevents API serving calls from accidentally polluting the training store, and allows the API container to run without any database dependency.

5. Step 2 — Data Ingestion and Storage

Directory: `step_2_ingestion/`

Key files: `ingest.py`, `schema.sql`, `spatial.py`, `compose.yaml`

5.1 Database Design

The MySQL schema is defined in `schema.sql` and consists of four tables:

```
stations      -- one row per monitoring station (id, name, province, lat/Lng, altitude)
sensors       -- one row per sensor (sensor id, station id, pollutant type, units, active period)
measurements  -- one row per (sensor, datetime) measurement (value, validation status)
weather_hourly -- one row per (station, datetime) hourly weather observation
```

The `measurements` table uses a composite unique key `(idsensore, data)` to make ingestion idempotent: re-running `ingest.py` on files already loaded will produce zero duplicate rows (the `INSERT IGNORE` strategy). The same idempotency applies to `weather_hourly` via its `(idstazione, dt)` unique key.

Why MySQL? The training pipeline requires flexible SQL aggregations (e.g., daily means, min, max across sensor types per station). MySQL is a proven, well-supported relational engine that integrates naturally with Python via `pymysql`. It is used exclusively as a **local development and training store** — the API container does not connect to it.

Docker Compose: `compose.yaml` starts a MySQL 8.0 container with the schema auto-executed at first start, a named volume for data persistence, and environment variables matching `ingest.py` defaults. This allows any developer to reproduce the full training pipeline with a single `docker compose up -d`.

5.2 ETL Process

The ingestion orchestrator (`ingest.py`) proceeds in four phases:

1. **Sensor registry:** fetches metadata from ARPA (or from a local cache file) and upserts all stations and sensors into MySQL. This establishes the foreign key relationships needed for measurement lookup.
2. **Measurements:** iterates over all `*_measurements.json` files in `data/raw/`, parses each record, and inserts via `INSERT IGNORE` in batches of 5,000 rows to balance transaction overhead and memory usage.
3. **Weather:** similarly iterates over `*_weather.json` files. Each file contains a per-station list of hourly time series. The loader flattens the nested JSON structure (one row per station-hour) and inserts into `weather_hourly`.
4. **Industrial proximity:** if `data/raw/industrial_zones.geojson` exists, `spatial.py` is called to compute `dist_industrial_km` for each station using GeoPandas in a projected metric CRS (EPSG:32632). The result is saved as `data/raw/industrial_proximity.parquet`.

5.3 Batch Insert Strategy

Rather than issuing one `INSERT` per record, the loader uses `executemany` with a `MYSQL_BATCH_SIZE = 5_000` parameter. This reduces the number of round-trips to MySQL by several orders of magnitude and dramatically accelerates ingestion of multi-year archives. The trade-off (slightly larger transactions) is acceptable because the ingestion is a background process and atomicity per-file is sufficient.

5.4 Spatial Feature Computation

`spatial.py` reads the station coordinates from MySQL and the industrial polygon boundaries from GeoJSON, both loaded as GeoDataFrames using GeoPandas. It then computes:

- `dist_industrial_km`: the minimum distance from each station point to the nearest industrial polygon boundary, using the projected CRS (EPSG:32632, UTM Zone 32N) for metric accuracy.

This feature represents a station-context proxy correlated with baseline emissions and complements the meteorological predictors. (A companion `n_industrial_zones_15km` count was evaluated and dropped — see §4.4.)

6. Step 3 — Exploratory Data Analysis and Feature Engineering

Directory: `step_3_edu/`

Key files: `build_dataset.py`, `eda.py`, `config.py`, `plots.py`, `rmt.py`

Output artifact: `daily_dataset_clean.parquet`

6.1 Dataset Construction

`build_dataset.py` is the central feature-engineering module. It reads from MySQL and produces a wide daily DataFrame at the grain of one row per (station, day). The construction pipeline is:

```
load_pm10_daily()      → raw PM10 per (station, day)
load_weather_daily_agg() → daily aggregated weather (mean/min/max/sum/count)
load_pollutants_daily_agg() → daily aggregated NO2, O3, CO, PM2.5 (mean/max)
_fill_date_gaps()      → ensures contiguous daily time series per station
merge_joins            → wide table: PM10 + weather + pollutants + industrial
_add_temporal_features() → month, season, weekday, heating season, cyclic encodings
_add_lag_and_rolling_features() → PM10 lags (1d, 2d) + rolling means (3d, 7d)
                                + weather lags (1d, 2d for pressure, wind, BLH, temp)
_add_alert_class()      → 4-class label from PM10 thresholds
_add_stagnation_flag()  → binary stagnation flag
_add_stagnation_index() → continuous stagnation index
_drop_redundant_features() → remove highly collinear columns
clean_dataset()         → drop sparse columns/stations, forward-fill pollutants/lags
```

6.2 Dataset Statistics

After construction and cleaning:

Dimension	Value
Total rows (station-days)	54,595
Monitoring stations	67
Date range	2024-01-01 → 2026-05-07
Feature columns (input to ML)	~50
PM10 distribution skew	~1.71
PM10 distribution kurtosis	~5.64

The strong positive skew of PM10 (skew ≈ 1.71 , kurtosis ≈ 5.64) is the key distributional fact motivating several design choices: the log-transform applied to regression targets, the class-balanced weights in classification, and the stagnation index as a domain-informed feature.

6.3 Feature Groups

The final feature set spans seven semantic groups:

Group	Features	Rationale
PM10 history	pm10_lag1 , pm10_lag2 , pm10_rol13 , pm10_rol17 , pm10_diff	PM10 exhibits strong autocorrelation; yesterday's value is the single best predictor
Meteorology	temp_mean , humidity_mean , dewpoint_mean , precip_sum , pressure_mean , cloud_cover_mean , wind_speed_mean , wind_speed_max , radiation_mean , blh_mean , blh_min , fog_hours	Atmospheric conditions directly control PM10 dilution, transport, and wet deposition
Meteorological lags	pressure_mean_lag1 , wind_speed_mean_lag1/lag2 , blh_mean_lag1/lag2 , temp_mean_lag1/lag2 , pressure_rol13 , wind_speed_rol13	Lagged meteorology captures persistence of anticyclonic regimes
Secondary pollutants	no2_mean , no2_max , o3_mean , o3_max , co_mean	Traffic-sourced NO ₂ and photochemical O ₃ co-vary with PM10 sources and sinks
Temporal	mese , stagione , giorno_settimana , is_weekend , heating_season , mese_sin , mese_cos , dow_sin , dow_cos	Strong seasonal and weekly patterns; cyclic encoding prevents ordinal artefacts
Spatial/spatial	quota , lat , lng , provincia , dist_industrial_km	Station location captures orographic and industrial exposure differences
Derived	stagnation_flag , stagnation_index	Engineered domain features capturing the main meteorological mechanism for PM10 accumulation

6.4 Temporal Features and Cyclic Encoding

Month and day-of-week are encoded both as raw integers and as cyclic sine/cosine pairs:

```
mese_sin = sin(2π × month / 12)
mese_cos = cos(2π × month / 12)
```

The cyclic encoding is essential for tree-based models and linear models alike: it ensures that December (month 12) and January (month 1) are represented as numerically close — which they are in terms of atmospheric conditions — rather than as the most distant integers in the ordinal sequence.

6.5 Stagnation Index

The stagnation index is a physics-inspired derived feature that quantifies the capacity of the atmosphere to trap pollutants near the surface:

```
stagnation_index = 1 / (wind_speed_mean × blh_min × (1 + precip_sum))
```

Justification: the higher the surface wind speed, the stronger the horizontal dilution. The deeper the boundary layer height (BLH), the larger the volume available for vertical mixing. Precipitation provides wet deposition, removing particles from the air column. The formula is dimensionless (once the denominator is clipped to physical minima) and produces a value in (0, 1] that rises monotonically with stagnation.

Wind speed and BLH are clipped to physical minima (`0.1 m/s` and `10 m` respectively) to prevent numerical instability under extreme calm/shallow-layer conditions.

6.6 Missing Value Strategy

The dataset contains missing values from three sources: (a) stations with sparse PM10 measurement coverage, (b) secondary pollutants available only at a subset of stations, and (c) weather aggregations failing when Open-Meteo returned incomplete hourly series.

The adopted strategy has two phases, strictly separated to prevent data leakage:

Pre-split cleaning (`clean_dataset`): 1. Drop any column with more than 50% missing values (threshold `MISSING_THRESHOLD = 0.50`). Protected columns (`idstazione` , `data_giorno` , `pm10` , `classe_allerta`) are never dropped. 2. Drop stations where valid PM10 records cover fewer than 50% of all study days. 3. Forward-fill pollutant and lag/rolling columns *within* each station group (past-only, no leakage): a missing NO₂ reading on day D is filled with the last known value for that station. 4. Drop rows with missing PM10 (the target).

Post-split imputation (`impute_missing`): 5. Compute column medians from the **training set only**. 6. Apply these training medians to fill remaining NaN in both train and test sets.

This two-phase approach is a strict leakage prevention measure. Using test-set statistics to impute training data (or vice versa) would give the model implicit knowledge of future values, inflating apparent performance.

6.7 Train/Test Split

All models use the same temporal split:

Split	Date range	Rows
Training	2024-01-01 → 2025-08-22	38,335
Test	2025-08-23 → 2026-05-07	16,260

The split point is at the **70th percentile of unique days** in the dataset, ensuring that all stations are represented in both splits (all 67 stations appear in both train and test). The temporal ordering is preserved: no test date appears before any training date. This reflects the real-world constraint that a model cannot use future data to predict past values.

6.8 Cross-Validation Strategy

Hyperparameter tuning uses a group-aware `TimeSeriesSplit` with 5 folds. The key property is that all rows from the same calendar day (across all 67 stations) are kept together in either the training or validation fold — never split across them. This prevents a subtle form of leakage where a model could implicitly learn from the PM10 values of other stations on the same day to predict a target station's value.

The CV splits are built by `shared/utils.py:make_temporal_cv_splits` , which: 1. Sorts training rows by `data_giorno` . 2. Partitions unique dates into folds. 3. Returns (`train_indices` , `val_indices`) tuples where indices correspond to rows, not dates.

6.9 EDA Findings

The EDA produces 16 plots and a set of statistical summaries. Key findings that directly informed modelling choices:

- 1. **PM10 autocorrelation**: correlation of PM10 with `pm10_lag1 ≈ 0.79` , `pm10_rol13 ≈ 0.71` , `pm10_rol17 ≈ 0.61` . This confirms that lagged PM10 is the dominant predictor, justifying the inclusion of lag and rolling

features.

2. **Strong seasonality:** winter PM10 is approximately twice the summer value (heating_season effect). The stagnation index tracks winter inversions closely.
3. **Meteorological drivers:** wind speed, BLH, and precipitation show a strong inverse relationship with PM10 — higher values (better dispersion) correspond to lower pollution. Pressure shows a positive association (anticyclonic conditions trap pollution).
4. **Non-Gaussian target:** PM10 has heavy right tails (kurtosis ≈ 5.64) with large peaks concentrated in the `rosso` class. This rules out ordinary linear regression without a target transformation.
5. **Province-level heterogeneity:** Bergamo, Brescia, and Cremona consistently show higher PM10 distributions, while Alpine-foothill stations (Como, Lecco) tend to be cleaner.
6. **Industrial proximity effect:** stations within 2 km of an industrial zone have a higher median PM10 (`+5-8  $\mu\text{g}/\text{m}^3$` in exploratory analysis), though the effect is partially confounded with urban density.

7. Step 3.5 — Random Matrix Theory Diagnostic

File: `step_3_eda/rmt.py`

Outputs: `rmt_summary.json`, `rmt_eigenvalues.csv`, `rmt_component_loadings.csv`, plots 14–16

7.1 Motivation

Pearson pairwise correlations are informative but give a myopic view of the feature space. Random Matrix Theory (RMT) provides a holistic alternative: it analyses the eigenspectrum of the full 41×41 feature correlation matrix and compares it against the theoretical noise floor derived from the **Marchenko-Pastur (MP) distribution**.

Under the MP null hypothesis (features are random noise), all eigenvalues of the correlation matrix of an $N \times p$ random matrix should fall within the interval $[\lambda^-, \lambda^+]$:

$$\lambda \pm = (1 \pm \sqrt{q})^2 \quad \text{where} \quad q = p / N$$

Eigenvalues above λ^+ indicate genuine statistical structure — correlated factors that cannot be explained by chance.

7.2 Feature Selection for RMT

41 numeric features are analysed. Excluded from the analysis: - Target and identifier columns: `pm10`, `classe_allerta`, `idstazione`, `nomestazione`, `comune`, `data_giorno` - Raw calendar ordinals: `mese`, `giorno_settimana` (redundant with their cyclic encodings, which are included)

The remaining features are standardised (z-scored) before constructing the correlation matrix.

7.3 Marchenko-Pastur Bounds

With $N = 54,595$ samples and $p = 41$ features:

$$\begin{aligned} q &= 41 / 54,595 \approx 0.00075 \quad (\text{extremely small} - \text{very wide data}) \\ \lambda^+ &= (1 + \sqrt{0.00075})^2 \approx 1.056 \\ \lambda^- &= (1 - \sqrt{0.00075})^2 \approx 0.946 \end{aligned}$$

The very small q means the MP upper bound $\lambda^+ \approx 1.056$ is barely above 1. Any eigenvalue above 1.056 is therefore statistically significant at the MP level.

7.4 Empirical Null Model

In addition to the MP theoretical bound, an empirical null is constructed using a **grouped circular shift** method:

- For each station group, each feature column is shifted by a random integer offset, independently of other features.
- This preserves the within-station temporal autocorrelation structure (unlike an i.i.d. column shuffle) while destroying the cross-feature alignment.
- 100 iterations produce a null distribution of maximum eigenvalues; the 95th percentile serves as a stricter threshold.

This null is more appropriate for environmental time series than an i.i.d. shuffle, because it accounts for the temporal structure of atmospheric measurements.

7.5 Results

Metric	Value
Features analysed	41
MP upper bound (λ^+)	1.056
Components above MP bulk	9
Variance explained by above-MP components	74.0%
Components above empirical p95 null	9

The agreement between the MP bound and the empirical null (both identify exactly 9 components) strongly corroborates the finding: the feature space contains 9 genuine structural factors, collectively explaining 74% of the spectral variance.

7.6 Interpretation of Principal Components

The component loadings (top absolute loadings per eigenvector) reveal three dominant factors:

Component	Eigenvalue	Dominant features	Interpretation
1	~11.8	<code>mese_cos</code> , <code>temp_mean*</code> , <code>heating_season</code> , <code>radiation_mean</code> , <code>o3_*</code>	Seasonal/photochemical cycle
2	~4.2	<code>pressure_*</code> , <code>lat</code> , <code>quota</code> , <code>pm10_roll*</code> , <code>pm10_lag*</code>	Persistence + geography (stagnation regimes)
3	~2.9	<code>wind_speed_*</code> , <code>wind_speed_roll3</code> , <code>stagnation_index</code> , <code>precip_sum</code>	Ventilation and wet deposition

This spectral decomposition provides independent corroboration of the domain knowledge encoded in the feature engineering choices: seasonality, persistence, and ventilation are the three main axes of PM10 variability in the dataset.

7.7 Relation to PCA and Modelling Pipeline

The RMT diagnostic uses the same spectral ingredients as PCA: the standardised feature correlation matrix is decomposed into eigenvalues and eigenvectors, and the eigenvector loadings are interpreted as principal components. The difference is methodological intent. Here the decomposition is used only as an **EDA diagnostic** to separate structured correlation factors from noise via the Marchenko-Pastur reference and the empirical null.

It is therefore **not** used as a dimensionality-reduction step before regression or classification. The modelling pipelines keep the original engineered features, then apply train-only imputation and model-specific preprocessing after the temporal split. This preserves feature interpretability, avoids PCA-related leakage, and is better aligned with tree-based models such as XGBoost and Random Forest, which can exploit non-linear interactions directly.

8. Step 4 — Regression

Directory: `step_4_regression/`

Key files: `config.py`, `train.py`, `evaluate.py`

8.1 Problem Formulation

The regression task predicts the next-day PM10 concentration in $\mu\text{g}/\text{m}^3$ (a continuous, strictly non-negative value) for each station, given all available past and concurrent features. This is a **temporal regression** problem: predictions must be made without access to future PM10 values.

8.2 Models Compared

Three models are trained and compared:

Model	Rationale	Preprocessing
ElasticNet	Linear baseline; interpretable; establishes the variance explainable by linear combinations	StandardScaler + log1p target transform
XGBoost Regressor	State-of-the-art gradient boosting; handles non-linearity, interactions, and missing values	Tree-mode (passthrough numerics) + log1p target transform
Random Forest Regressor	Ensemble baseline; less prone to overfitting than boosting; slower but more stable	Tree-mode + log1p target transform

8.3 Preprocessing Pipeline

Each model is wrapped in a scikit-learn `Pipeline` with two stages:

- ColumnTransformer** (`build_preprocessor` from `shared/utils.py`): - For linear models (`model_type="linear"`): `StandardScaler` on all numeric columns; `OrdinalEncoder` on categoricals (`provincia`, `stagione`). - For tree models (`model_type="tree"`): numeric columns passed through (trees are scale-invariant); `OrdinalEncoder` for categoricals.
- TransformedTargetRegressor** wrapping the base estimator: - `func = np.log1p` applied to `y` before fitting. - `inverse_func = np.expm1` applied to predictions to recover the original scale.

Justification for log-transform on target: PM10 is right-skewed with a long tail of high-pollution peak days. Training a model with RMSE loss in the original scale would disproportionately penalise errors on already-rare

peak events, while the model simultaneously tries to minimise the much larger volume of low-moderate pollution days. The log-transform compresses the tail, effectively giving peak days proportionally more weight relative to their magnitude. The final predictions are always in the original $\mu\text{g}/\text{m}^3$ scale.

8.4 Hyperparameter Tuning

Model	Search method	Grid size	CV fits
ElasticNet	GridSearchCV	$6 \alpha \times 4 \text{ l1_ratio} = 24$	$24 \times 5 = 120$
XGBoost	RandomizedSearchCV	50 from $4 \times 4 \times 3 \times 3 \times 3 \times 3 = 1,296$	$50 \times 5 = 250$
Random Forest	RandomizedSearchCV	50 from $3 \times 3 \times 3 = 27$	$50 \times 5 = 250$

The exhaustive grid search for ElasticNet is feasible because ElasticNet fits are extremely fast (each fold takes < 1 second on a modern CPU). For XGBoost and Random Forest, the full grid would require thousands of fits; RandomizedSearchCV samples 50 combinations ($\approx 4\%$ of the XGBoost space), which empirically captures $\sim 90\%$ of the quality of an exhaustive search at a fraction of the computational cost.

Scoring metric: `neg_root_mean_squared_error` (i.e., `-RMSE`). RMSE is chosen over MAE because it penalises large errors more heavily, which is appropriate for a domain where peak PM10 days are the most consequential.

8.5 Regression Results

Model	RMSE ($\mu\text{g}/\text{m}^3$)	MAE ($\mu\text{g}/\text{m}^3$)	R^2
<code>elasticnet</code>	16.60	7.92	0.018
<code>random_forest</code>	9.39	6.39	0.686
<code>xgboost</code>	9.25	6.24	0.695

The ElasticNet performance ($R^2 \approx 0.018$) reveals that the linear model almost entirely fails to capture the non-linear structure of the problem. This is consistent with EDA finding that the dominant drivers (lag interactions, stagnation thresholds, province $\times$ season interactions) are inherently non-linear. ElasticNet is retained for interpretability benchmarking, but is not suitable for production.

RMSE by alert class:

Model	Verde	Giallo	Arancio	Rosso
<code>elasticnet</code>	6.11	6.51	12.21	41.00
<code>random_forest</code>	5.92	7.02	9.26	18.16
<code>xgboost</code>	5.62	7.01	9.17	17.93

The most striking result is ElasticNet's RMSE of $41 \mu\text{g}/\text{m}^3$ in the `rosso` class — nearly double that of XGBoost (17.93). This confirms that linear models systematically under-predict pollution peaks. XGBoost dominates in every class, with the largest advantage precisely where it matters most (high PM10 days).

8.6 Feature Importance

Permutation importance (computed on the test set with 10 repeats, scoring by R^2) shows:

- `pm10_rol17` — by far the most influential feature (long-run PM10 persistence)
- `pm10_lag1` — yesterday's PM10 value

3. `pressure_mean` — anticyclonic regime indicator
4. `o3_mean` — photochemical co-variable (summer-winter signal)
5. `provincia` — captures station-level fixed effects
6. `stagnation_index` — engineered dispersion capacity
7. `blh_mean` — boundary layer height (mixing depth)

9. Step 5 — Classification

Directory: `step_5_classification/`

Key files: `config.py`, `train.py`, `evaluate.py`, `ordinal_classifier.py`, `calibrate.py`, `tune_hybrid.py`, `guardrails.py`

9.1 Problem Formulation

The classification task predicts the next-day alert class as a 4-class ordinal variable. The label encoding preserves ordinality:

```
LABEL_MAP = {"verde": 0, "giallo": 1, "arancio": 2, "rosso": 3}
```

Why not just threshold the regression output? Thresholding the XGBoost regressor output on the PM10 thresholds [20, 35, 50] is a valid baseline (`regression_to_class_xgboost`), but a dedicated classifier can optimise directly for classification metrics and apply class-specific weighting, which is important given the class imbalance.

Class distribution (test set):

Class	Count	Fraction
<code>verde</code>	5,555	34.2%
<code>giallo</code>	5,445	33.5%
<code>arancio</code>	3,135	19.3%
<code>rosso</code>	2,125	13.1%

The distribution is moderately imbalanced, with the `rosso` class being the rarest. This motivates the use of class-balanced weights and the introduction of a production readiness criterion specifically targeting `recall_rosso`.

9.2 Production Readiness Criteria (Hard Constraints)

For any classification strategy to be considered production-ready — suitable for use in a public early-warning system — two hard constraints must be satisfied simultaneously:

- **C1:** `recall_rosso ≥ 0.65` — at least 65% of true red-alert days must be correctly identified. A missed red alert (false negative) has a higher cost than a false alarm: it could prevent protective measures from being taken on a heavily polluted day.
- **C2:** `severe_error_rate ≤ 2.5%` — at most 2.5% of all predictions may be off by two or more classes (e.g., predicting `verde` when the true class is `arancio` or `rosso`). These "gross errors" are the most dangerous from a public health perspective.

9.3 Models and Strategies

The classification step evolves through three phases:

Phase 1 — Baseline classifiers:

Model	Handling of class imbalance	CV scoring
Logistic Regression	<code>class_weight="balanced"</code> + ElasticNet regularisation	F1-macro
Random Forest	<code>class_weight="balanced"</code>	F1-macro
XGBoost	<code>compute_sample_weight("balanced", y_train)</code>	F1-macro

Phase 2 — Hybrid strategy (regression × classification): The XGBoost regressor and XGBoost classifier are combined: if the regressor's predicted PM10 exceeds a threshold δ above the alert boundary *and* the classifier's probability for a higher class exceeds `p_threshold`, the higher class is assigned. Parameters (δ , `p_threshold`) are optimised on out-of-fold (OOF) predictions with a composite objective:

```
objective =  $\alpha$  × F1_macro +  $\beta$  × recall_rosso - severe_error_rate
```

where $\alpha = 1.0$, $\beta = 0.5$. The optimal parameters found are $\delta = 7.0$, `p_threshold = 0.25`.

Phase 3 — Ordinal classifier (Frank & Hall 2001): The `OrdinalClassifier` (implemented in `ordinal_classifier.py`) decomposes the 4-class ordinal problem into 3 binary classifiers: - Classifier 1: $P(y > 0)$, i.e., $P(\text{not verde})$ - Classifier 2: $P(y > 1)$, i.e., $P(\text{not verde and not giallo})$ - Classifier 3: $P(y > 2)$, i.e., $P(\text{rosso})$

Class probabilities are reconstructed as:

```
P(y=0) = 1 - P(y>0)
P(y=k) = P(y>k-1) - P(y>k)    for 0 < k < K-1
P(y=K-1) = P(y>K-2)
```

Each binary classifier receives independent balanced sample weights, because the class imbalance ratio differs across the three sub-problems ($P(y > 2)$ is much more skewed than $P(y > 0)$). Results are clipped to [0,1] and renormalised to sum to 1 to handle numerical imprecision.

Why this approach? The confusion matrix from Phase 1 classifiers showed that most misclassifications occur between adjacent classes (verde/giallo, giallo/arancio), which is consistent with an ordinal structure. The Frank & Hall decomposition explicitly exploits this structure by learning monotone probability estimates.

9.4 Probability Calibration

The best Phase 1 classifier (XGBoost) is calibrated using isotonic regression with temporal CV splits (`CalibratedClassifierCV(method="isotonic", cv=tscv)`). Calibration is fitted on the training set only to prevent test-set contamination.

Expected Calibration Error (ECE) before and after calibration (10 bins):

Class	ECE before	ECE after
verde	0.0363	0.0447
giallo	0.0433	0.0318
arancio	0.0513	0.0289
rosso	0.0103	0.0083

The `verde` class shows a slight degradation post-calibration — this is acceptable because `verde` is the majority class and its overconfidence is less harmful. The critical classes (`arancio`, `rosso`) both improve, which is the primary objective of calibration.

The calibrated model is used as the base for the hybrid strategy (Phase 2), because the hybrid decision rule relies on class probabilities.

9.5 Complete Results

Strategy	F1-macro	Recall rosso	Severe error rate	C1	C2	Production ready
<code>logistic_regression</code>	0.616	0.657	2.79%	✓	✗	NO
<code>random_forest</code>	0.634	0.577	2.12%	✗	✓	NO
<code>xgboost</code>	0.639	0.618	1.94%	✗	✓	NO
<code>regression_to_class_xgboost</code>	0.625	0.464	1.93%	✗	✓	NO
<code>xgboost_ordinal</code> (Phase 3)	0.619	0.803	2.97%	✓	✗	NO
<code>hybrid_xgboost_reg_xgboost_cls</code> (Phase 2)	0.636	0.680	2.32%	✓	✓	YES

Only the hybrid strategy passes both hard constraints.

9.6 Selecting the Deployment Model

Only the hybrid strategy successfully passes both hard constraints, making it the unique production-ready choice.

- `xgboost_ordinal` maximises `recall_roso` (0.803), which makes it the most conservative choice, but its `severe_error_rate` (2.97%) exceeds the maximum allowable budget of 2.5% (C2). This degradation is driven by severe false alarms: it predicts `rosso` on actual `giallo` days 234 times (vs 148 for the hybrid) and `arancio` / `rosso` on actual `verde` days 144 times (vs 63 for the hybrid). For a public agency, such severe over-predictions carry a high administrative and political cost (e.g., unjustified traffic restrictions) and risk eroding public trust (the "cry wolf" effect).
- `hybrid_xgboost_reg_xgboost_cls` achieves the best overall balance: a strong `f1_macro` (0.636 — within 0.003 of the top base classifier, which itself fails C1), a solid `recall_roso` (0.680), and safely passes the C2 constraint with a `severe_error_rate` of 2.32% (limiting high-severity false alarms to 63 for green and 148 for yellow).

The API serves the hybrid strategy because it is the only model providing strong holistic performance while strictly respecting the public health and reliability guardrails. The ordinal model is also saved for potential high-sensitivity (but higher false-alarm) scenarios.

10. Step 6 — Station Clustering

Directory: `step_6_clustering/`

Key files: `config.py`, `profiles.py`, `cluster.py`, `evaluate.py`, `map_view.py`

10.1 Objectives and Scope

The clustering step performs **unsupervised learning on the 67 monitoring stations** (not on individual station-days). The goal is to identify typologies of stations based on their long-run PM10 profile, without using any labels from the supervised tasks. This analysis:

- Provides independent validation that the dataset captures meaningful, persistent structure across stations.
- Helps identify which station types are most at risk and may require targeted monitoring.
- Enriches the API response with a cluster label per station.

10.2 Station Profile Construction

Each of the 67 stations is reduced to an 8-dimensional aggregate profile by collapsing all station-day rows (across the full study period) into a single row per station:

Feature	Aggregation	Rationale
pm10_mean	Mean	Chronic pollution level
pct_critical_days	Fraction of days with classe_allerta ∈ {arancio, rosso}	Severity beyond the median
seasonality_ratio	Winter PM10 mean / Summer PM10 mean	Seasonal amplitude (heating + inversion)
stagnation_index_mean	Mean	Long-run stagnation exposure
dist_industrial_km	First (static)	Proximity to industrial sources
no2_mean	Mean	Traffic/combustion proxy
quota	First (static), imputed for 1 missing station	Topographic dilution
wind_speed_mean	Mean	Long-run dispersion capacity

Design decisions: - **O₃ excluded:** only 37/67 stations measure O₃. Including it would require imputing the 45% gap, which would introduce synthetic variability and distort the distance metric. NO₂ (available at 66/67 stations, coverage ≈ 99%) is retained. - **pct_critical_days over class-dominance:** the dominant alert class per station is verde for 50/67 stations, giallo for 17, and arancio / rosso for 0. This collapses to a highly imbalanced binary variable with little discriminative power. pct_critical_days provides a continuous severity measure with much higher entropy. - **RobustScaler:** preferred over StandardScaler because a few urban-industrial outlier stations have extreme PM10/NO₂ values that would otherwise dominate the Euclidean distance metric.

10.3 Algorithms Evaluated

Three algorithms are evaluated across a range of cluster counts:

Algorithm	Parameter range	Justification
KMeans	k ∈	Fast, deterministic, minimises within-cluster variance
Agglomerative Ward	k ∈	Hierarchical; dendrogram gives visual insight
DBSCAN	ε ∈ {0.5, 0.75, 1.0, 1.25, 1.5}, min_samples=3	Density-based; included as comparison only

A partition is considered **non-degenerate** if every cluster contains at least 25% of the expected balanced size (floor: 3 stations). Degenerate partitions (where almost all stations collapse into one cluster) are excluded from the model selection criterion.

10.4 Results

Algorithm	k	Silhouette	Calinski-Harabasz	Davies-Bouldin	Non-degenerate
KMeans	2	0.467	33.4	1.134	✓
KMeans	3	0.358	39.0	0.994	X (degenerate)
Agglomerative Ward	2	0.694	33.2	0.507	X (sizes [65, 2])
DBSCAN	$\epsilon=0.75$	0.407	—	—	X (55 noise points)

KMeans k=2 is selected as the best model: it is the only non-degenerate partition with silhouette > 0.4. Agglomerative Ward k=2 has a higher silhouette on paper (0.694) but is degenerate — 65 of 67 stations end up in one cluster, which has no interpretive value. DBSCAN at every tested ϵ value assigns the majority of stations to the noise class, confirming that the station population does not exhibit compact density clusters in 8-dimensional scaled space.

10.5 Cluster Profiles

Cluster	N stations	Description	Characteristics
Cluster 0	56	"Typical" stations	Lower PM10 mean, fewer critical days, better ventilation
Cluster 1	11	"High-pollution" stations	Elevated PM10, more critical days, lower wind, closer to industrial zones

10.6 Validation

External validation vs. province (independent of feature set):

Metric	Value	Interpretation
Adjusted Rand Index	0.038	Clusters do not correspond to administrative provinces
Homogeneity	0.119	Clusters are not pure in terms of province
Completeness	0.644	Stations from the same province tend to land in the same cluster
V-measure	0.201	Balanced harmonic mean

The low ARI is expected and desirable: the clustering captures **pollution regimes** (industrial exposure + topography + wind patterns), not administrative geography. The high completeness (0.644) indicates that province-level geographic co-location is a partial predictor of cluster membership, but not the primary driver.

Statistical separation (Kruskal-Wallis non-parametric test):

Feature	Statistic	p-value
pm10_mean	24.26	8.4×10^{-7}
pct_critical_days	26.13	3.2×10^{-7}

Both key cluster-defining features show extremely highly significant differences between clusters ($p < 10^{-6}$), confirming that the partition captures real, statistically robust structure rather than noise artefacts.

Circularity caveat: cluster features include `pm10_mean` and `pct_critical_days`, which are derived from the same variable (`pm10`) used to construct `classe_allerta`. Validating clusters against `classe_allerta`-derived labels would not be independent. The primary external validation therefore uses `provincia` (geographic label, independent of PM10). The confusion-matrix-based validation against `classe_allerta` is reported only for completeness and is explicitly marked `"non_independent": true` in `clustering_metrics.json`.

11. REST API and Web Interface

Directory: `api/`

Key files: `main.py`, `schemas.py`, `routers/`, `services/`

11.1 Architecture Principles

The API is designed around two principles:

1. **No database dependency at runtime:** the serving container has no connection to MySQL. All data needed for prediction (station metadata, historical PM10 statistics) is read from `daily_dataset_clean.parquet` at startup. Recent data (last 7 days, needed for lag features) is fetched on demand from GCS.
2. **Single-container deployment:** the FastAPI backend and the Jinja2 HTML frontend are served by the same ASGI application, which can be deployed as a single Docker container on Cloud Run.

11.2 API Endpoints

Endpoint	Method	Description
<code>/health</code>	GET	Liveness check → <code>{"status": "ok"}</code>
<code>/stations</code>	GET	List all 67 stations with name, municipality, coordinates
<code>/forecast</code>	GET	Next-day (or day+2) PM10 forecast + alert class for a station
<code>/history</code>	GET	Historical PM10 and alert class for a station and date range
<code>/api/clusters</code>	GET	Cluster label and profile for all stations (JSON)
<code>/clusters</code>	GET	HTML page: display cluster profiles and station assignments
<code>/</code>	GET	Interactive map (Folium, server-side rendered)
<code>/forecast-ui</code>	GET/POST	HTML form: select station and horizon, display forecast
<code>/history-ui</code>	GET/POST	HTML form: select station and period, display Plotly chart

11.3 Prediction Pipeline (`services/predictor.py`)

At startup, all model artefacts (`.joblib` files) are loaded once into module-level variables. Per-request prediction proceeds as follows:

1. **Fetch recent data:** `services/recent_data.py` fetches the last 7 days of measurements from GCS (with a 15-minute TTL cache). Sentinel values (`valore ≤ 0`) are replaced with `NaN`.
2. **Extract lag features:** from the cached DataFrame, compute `pm10_lag1`, `pm10_lag2`, `pm10_rol17`, `pm10_rol13` for the requested station. A helper `_last_valid(series)` uses `dropna()` to robustly handle gaps.
3. **Fetch forecast weather:** `services/weather.py` calls the Open-Meteo live forecast API for the station's coordinates and aggregates hourly values to daily (mean/sum/min/max using the same aggregation rules as `build_dataset.py`).
4. **Assemble feature vector:** `build_features()` constructs the full feature vector combining weather forecast, lags, static station metadata, and temporal features. The day+2 forecast uses the day+1 predicted PM10 as `pm10_lag1`, propagating uncertainty.
5. **Predict:** the regression model produces a PM10 estimate; the hybrid classification strategy produces an alert class and probability vector.
6. **Data quality assessment:** `compute_data_quality()` evaluates whether the lag data window has sufficient valid days. If fewer than 3 valid days are available (`data_quality = "stale"`), an HTTP 422 error is returned rather than a potentially unreliable prediction.

11.4 Pydantic Schemas

All API responses are validated by Pydantic v2 models:

```
class ForecastItem(BaseModel):
    date: date
    pm10_predicted: float
    alert_class: str
    alert_index: int
    weather_used: WeatherUsed
    provisional: bool = False
    provisional_reason: Optional[str] = None

class ForecastResponse(BaseModel):
    station_id: str
    station_name: str
    predictions: list[ForecastItem]
    data_quality: Literal["ok", "partial", "stale"]
    provisional: bool = False
    provisional_days: int = 0
```

The `provisional` field flags whether any lag/roll input was sourced from the live ARPA fallback (Socrata) rather than the validated GCS archive. This transparency is exposed both in the JSON API and in the HTML frontend.

11.5 Web Frontend

Three HTML pages are served via Jinja2 templates:

- **Map** (`/`): a server-side rendered Folium map of Lombardy with one colour-coded marker per station. Marker colour reflects the predicted alert class for day+1. Stations with `data_quality ≠ "ok"` display a visual indicator.
- **Forecast** (`/forecast-ui`): a form allowing the user to select a station and forecast horizon (1 or 2 days). The result page displays the predicted PM10 value, the alert class badge, the meteorological inputs used, and the station's cluster membership. If the prediction is provisional, a warning badge (⚡) is shown.

- **History** (`/history-ui`): a form for browsing historical PM10 data. Results are rendered as a Plotly time series with horizontal bands marking the four alert thresholds, allowing visual assessment of the station's pollution history.

12. Cloud Deployment

File: `Dockerfile`

12.1 Container

The production container is a minimal `python:3.11-slim` image. It copies only the files required for serving (no training code, no raw data files):

```
COPY api/                ./api/
COPY shared/             ./shared/
COPY step_3_eda/          ./step_3_eda/      # parquet + config (no MySQL dependency)
COPY step_4_regression/artifacts/ # regression .joblib files
COPY step_5_classification/artifacts/ # classification .joblib files
COPY artifacts/          ./artifacts/      # cluster artefacts
COPY data/raw/sensors_registry.json    # static station list
```

The entry point is `uvicorn api.main:app --host 0.0.0.0 --port ${PORT}`.

12.2 Cloud Run Configuration

- **Region:** `europe-west1` (Belgium) — geographically close to Lombardy, minimises GCS access latency.
- **Concurrency:** 80 requests per instance (FastAPI ASGI is non-blocking).
- **Min instances:** 1 (eliminates cold-start latency for a public-facing service).
- **Memory:** 1 GiB (sufficient for loading three `.joblib` models and the parquet in memory).
- **GCS access:** the Cloud Run service account has `storage.objectViewer` IAM role on the bucket.

12.3 Cloud Run Jobs

Two background jobs complement the main serving container:

Job	Schedule	Action
<code>daily-refresh</code>	Daily at 03:00 Europe/Rome	Refreshes the last 7 days of GCS blobs (force-overwrites last 3 days)
<code>backfill-heal</code>	Saturdays at 02:00 Europe/Rome	Re-downloads and overwrites the <code>[today-21, today-15]</code> window with validated ARPA data

13. Data Quality and Freshness Management

13.1 The ARPA Validation Lag

ARPA Lombardia marks freshly uploaded measurements as `stato = 'NA'` with a sentinel value of `-9999` when the automated quality control has not yet run. Full validation (`stato = 'VA'`) typically arrives 8–12 working days after the measurement date.

This creates a tension between: - **Freshness**: the API needs recent data (within the last 7 days) to compute lag features. - **Accuracy**: preliminary measurements may be spurious (sensor malfunctions, transmission errors).

13.2 Multi-Layer Defence

The system manages this tension through a five-layer defence:

- 1. **Sanitisation at ingestion** (`gcloud_daily/api_client.py`): any record with `valore ≤ 0` or `stato ≠ 'VA'` is stored with `valore = null` in GCS. The blob is complete (metadata preserved) but the measurement value is missing rather than misleading.
- 2. **Weekly heal job** (`gcloud_backfill/` , `OVERWRITE_EXISTING=true`): every Saturday, the `[today-21, today-15]` window is re-downloaded with `stato = 'VA'` only and overwrites the preliminary blobs. By this point ARPA has validated these days.
- 3. **Belt-and-suspenders filter at serving** (`recent_data.py`): even if a stale blob reaches the serving layer, all values ≤ 0 across all pollutants are replaced with `NaN` before feature extraction.
- 4. **Data quality score** (`compute_data_quality`): the API computes `valid_days_last_7` (number of days with at least one non-null PM10 reading) and classifies stations as `ok` ($\geq 6/7$ valid days), `partial` (3–5/7), or `stale` ($< 3/7$). Stale stations return HTTP 422 instead of an unreliable prediction.
- 5. **Socrata live fallback** (`live_arpa.py`): for days with no valid data in GCS, the API silently queries the Socrata endpoint for validated data (`stato = 'VA'`), which has a shorter validation lag for some pollutants. Predictions based on this fallback are flagged `provisional: true`.

14. Results Summary

14.1 Dataset

Metric	Value
Training samples	38,335
Test samples	16,260
Stations	67 (all in both splits)
Training period	2024-01-01 → 2025-08-22
Test period	2025-08-23 → 2026-05-07

14.2 Regression (Best: XGBoost)

Metric	Value
RMSE	9.24 µg/m³
MAE	6.26 µg/m³
R²	0.696
RMSE on <code>rosso</code> class	17.83 µg/m³

14.3 Classification (Best production-ready: Hybrid XGBoost)

Metric	Value
F1-macro	0.636 (best production-ready)
Recall rosso	0.680
Severe error rate	2.32%
Production ready	YES

Metric	xgboost_ordinal
F1-macro	0.619
Recall rosso	0.803 (best)
Severe error rate	2.97%
Production ready	NO

14.4 Clustering (Best: KMeans k=2)

Metric	Value
Silhouette	0.467
Cluster sizes	56 / 11
Kruskal-Wallis p (pm10_mean)	8.4×10^{-7}
ARI vs. province	0.038 (expected: clusters $\neq$ provinces)

14.5 RMT Diagnostic

Metric	Value
Features analysed	41
MP upper bound λ^+	1.056
Significant components	9
Variance explained	74.0%

15. Design Decisions Log

This section consolidates the non-obvious design choices made throughout the project, each with a brief justification.

Decision	Rationale
Temporal train/test split at 70th percentile of days	Ensures a clean temporal boundary; avoids future-data leakage; all stations appear in both splits
Group-aware TimeSeriesSplit CV	Prevents same-day station data leaking across train/val folds
Log1p target transform for regression	PM10 is right-skewed (kurtosis ≈ 5.64); log-space RMSE reduces systematic under-prediction of peaks

Decision	Rationale
Two-phase imputation (clean_dataset + impute_missing)	Strict separation of pre-split cleaning from post-split median imputation prevents test-to-train leakage
Forward-fill for pollutants (not global imputation)	Respects the direction of time; a missing NO ₂ value is filled with the station's last known reading
OrdinalEncoder for categoricals in tree models	Tree models handle arbitrary integer encodings; OrdinalEncoder is computationally cheaper than one-hot and does not inflate the feature space
F1-macro as CV scoring metric for classification	Penalises both false positives and false negatives equally across all four classes; appropriate for class-imbalanced multiclass problems
class_weight="balanced" + compute_sample_weight	Prevents the majority class (verde) from dominating training; XGBoost requires explicit sample weights instead of class_weight parameter
Hard constraints C1/C2 for production readiness	F1-macro alone cannot guarantee safety: a model with high F1-macro but low recall_rosso would miss red-alert days
Frank & Hall ordinal decomposition	Explicitly exploits the ordinal structure of the alert classes; each binary classifier is trained with class-specific weights tailored to that sub-problem's imbalance
Isotonic calibration with temporal CV splits	Isotonic regression is non-parametric and handles the non-monotone calibration errors of XGBoost; temporal CV prevents test-set contamination during calibration
RobustScaler for clustering	Median/IQR scaling is robust to the few extreme outlier stations (very high PM10/NO ₂) that would otherwise dominate Euclidean distance
KMeans preferred over Agglomerative at k=2	Agglomerative Ward k=2 produced a degenerate partition (65+2 stations); KMeans k=2 is the only non-degenerate option above silhouette 0.4
O₃ excluded from clustering profiles	Only 37/67 stations have O ₃ data; imputing 45% of a feature introduces more synthetic variance than the feature provides discriminative power
GCS for serving, MySQL for training	Complete isolation of training store (immutable archive) from serving path (ephemeral rolling window); API container requires no database
Grouped circular shift as RMT null	Destroys cross-feature correlations while preserving within-station temporal autocorrelation; more realistic null than i.i.d. column shuffle for environmental time series
Instantaneous PM10 thresholds (not consecutive-day rule)	Produces a balanced, classifiable target; the Lombardy anti-smog DGR 449/2018 consecutive-day rule would produce an extremely rare positive class unsuitable for ML

End of Technical Report